International Journal for Multidisciplinary Research (IJFMR)
E-ISSN: 2582-2160 ● Website: www.ijfmr.com ● Email: editor@ijfmr.com

# Prompt Engineering: Techniques, Failures, and Evaluations—A Comprehensive Survey

**Disha Nagpure*1, Atharv Sabde*2, Tanieshk Yadav*3, Pranita Dhoble*4, Soham Dhanduke*5**
[1]HoD, Dept. of AIML, ACEM, Pune, Maharashtra, India,
[2,3,4,5]Student, Dept. of AIML, ACEM, Pune, Maharashtra, India.

## ABSTRACT

Large language models (LLMs) have revolutionized natural language processing, with prompt engineering emerging as a critical factor in their effectiveness. This literature survey provides a comprehensive overview of current research on prompt engineering and failure modes in LLMs, systematically categorizing and organizing existing work in this rapidly evolving field. We explore the landscape of currently available prompting techniques, examining their design principles, applications, and effectiveness across diverse tasks. The survey organizes existing research into a systematic taxonomy of prompt failures, including content-related, structural, behavioral, and consistency issues as identified across multiple studies. We compile and classify assessment frameworks reported in the literature, encompassing automated metrics (BLEU, ROUGE, BERT Score), human evaluation protocols, and multidimensional quality measures. The survey maps optimization approaches documented across recent research—including genetic algorithms, gradient-based methods, and reinforcement learning techniques—illustrating the field's progression from empirical practices toward systematic methodologies. By comprehensively organizing and classifying the current body of research, this survey serves as a reference guide for researchers and practitioners seeking to understand the landscape of prompt engineering, failure modes, and optimization strategies in LLM applications.

**Keywords**- Prompt Engineering, Large Language models, Techniques, Failure cases, Evaluations, Survey.

## 1. INTRODUCTION

### a. Background on LLMs and Prompt Engineering

Prompt engineering designs input instructions to guide model behavior without modifying parameters. It's significantly more economical than traditional machine learning approaches—prompt iterations cost cents versus thousands for model fine-tuning, making it the primary interface for deploying LLMs in production.

Beyond cost, well-crafted prompts (often just a few hundred tokens) effectively guide billion-parameter models, reducing operational expenses by optimizing token consumption and streamlining outputs. However, prompting strategies don't universally transfer across model architectures due to architectural dependencies. The field has evolved from informal best practices to systematic research examining cognitive frameworks, optimization algorithms, and evaluation methodologies, though gaps remain in understanding prompt failures and improvement strategies.

**b. Problem Statement**

Prompt engineering research exhibits a fragmented landscape with several critical challenges hindering systematic understanding and practical application. Studies document various prompt failure modes—hallucinations, constraint violations, inconsistent outputs—but propose different classification schemes based on diverse theoretical perspectives (cognitive science, information theory, software engineering) without systematic organization.

Research reports numerous prompting techniques including zero-shot, few-shot, chain-of-thought, and role-based approaches, yet their documented effectiveness remains scattered across individual studies. Developers spend 60-80% of time in trial-and-error cycles, highlighting the gap between available techniques and practical guidance.

Literature also presents diverse evaluation approaches—automated metrics, human protocols, benchmark suites—but lacks comprehensive comparison of their applicability and limitations across different contexts.

This fragmentation makes it difficult for researchers and practitioners to understand the current state of knowledge, compare findings across studies, or identify areas requiring further investigation. A systematic survey is needed to compile and organize existing research, categorize documented techniques and failure modes, and map the landscape of evaluation approaches reported in the literature.

**c. Objectives**

This literature survey systematically compiles and organizes existing research on prompt engineering, structured around two primary objectives:

Objective 1: To systematically compile and organize prompt engineering techniques, failure classifications, and evaluation methodologies documented in literature. This survey catalogs prompting strategies including zero-shot, few-shot, chain-of-thought, tree-of-thoughts, role-based prompting, and tool-augmented approaches; compiles failure taxonomies across dimensions of specification, input, structure, context, performance, and maintainability; and maps evaluation frameworks including automated metrics (BLEU, ROUGE, BERT Score), human evaluation protocols, and benchmark suites with their reported effectiveness and limitations.

Objective 2: To identify patterns, gaps, and future directions in prompt engineering research. This includes documenting architectural dependencies, optimization cost variations, and domain-specific requirements; analyzing convergence and divergence across taxonomic frameworks; and cataloging research gaps in multimodal optimization, theoretical foundations, evaluation standardization, cross-model transferability, and real-world deployment validation.

### d. Significance and Scope

This survey provides a comprehensive reference for researchers and practitioners deploying LLMs. For researchers, it organizes taxonomies and evaluation frameworks with standardized terminology. For practitioners across healthcare, legal, education, finance, and customer service, it compiles failure patterns and mitigation strategies from published research.

Scope: The survey covers research from 2020 to early 2025, focusing on unintentional prompt failures in good-faith design efforts. Papers were selected for substantive contributions to understanding failure modes, evaluation methodologies, or systematic techniques advancing both theory and practice.

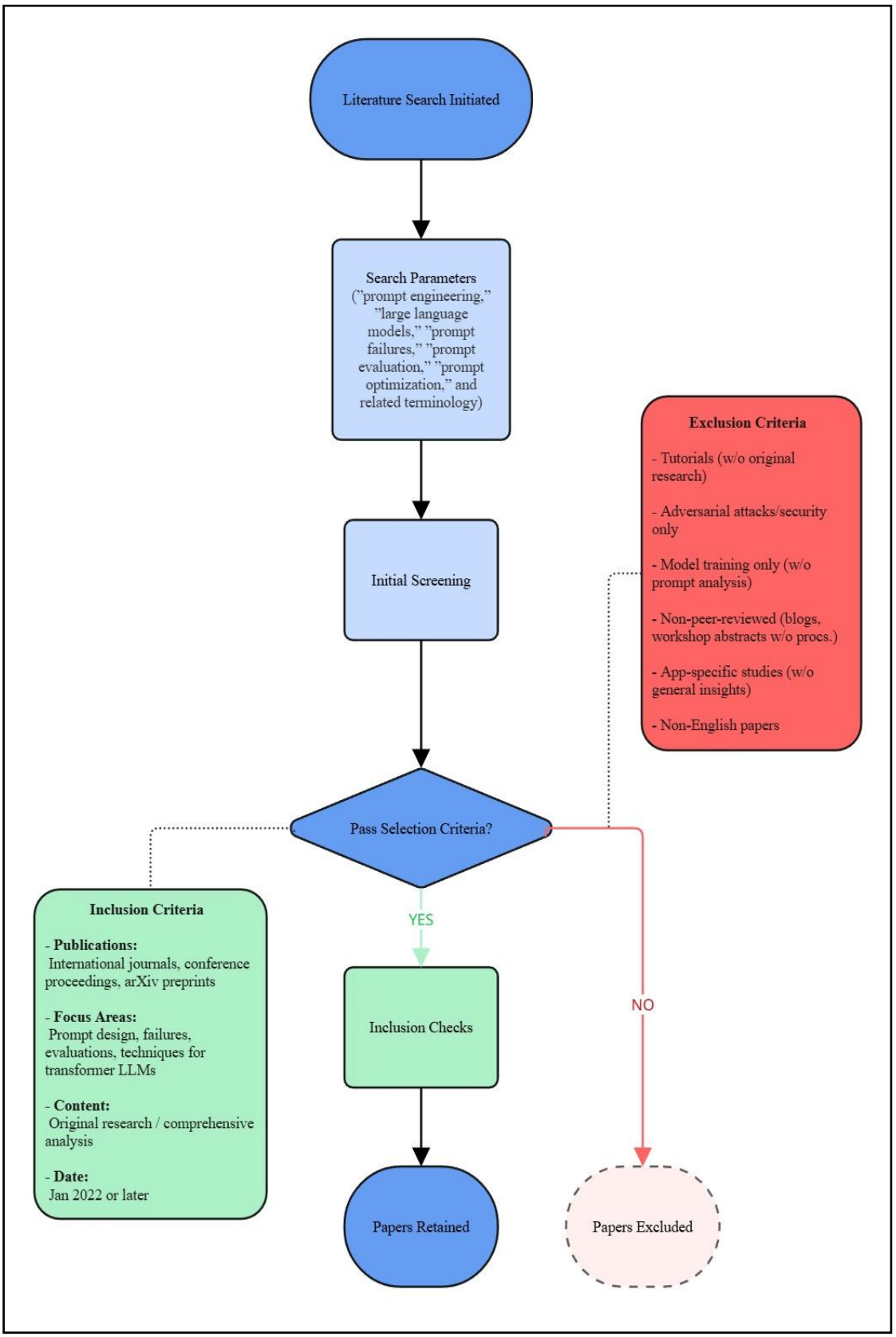
## 2. METHODOLOGY

A systematic literature search was conducted in September 2025 across multiple academic databases to ensure comprehensive coverage of prompt engineering research. The primary databases included Google Scholar, ACM Digital Library, arXiv (specifically the cs.CL, cs.AI, and cs.LG categories), and IEEE Xplore. These databases were selected for their comprehensive coverage of computer science and artificial intelligence research, including both peer-reviewed publications and preprints.

The search employed combinations of key terms including “prompt engineering,” “large language models,” “prompt failures,” “prompt evaluation,” “prompt optimization,” and related terminology. Example search strings included:

- “prompt engineering” AND “large language models”
- “prompt optimization” OR “prompt tuning”
- “prompt failures” OR “prompt defects” OR “prompt errors”
- “chain-of-thought” AND “reasoning”
- “few-shot learning” AND “prompting”
- “prompt evaluation” OR “prompt assessment” OR “prompt quality”
- “zero-shot” AND “in-context learning”
- “retrieval augmented generation” AND “prompting”

Figure 1: Systematic Selection Process Flow Diagram



Boolean operators (AND, OR) were used to capture relevant variations and combinations of these terms. The temporal scope covered publications from January 2020 to September 2025, a period characterized by rapid advancement in instruction-tuned LLMs and the emergence of prompt engineering as a distinct research area.

Selection Process: Papers underwent a multi-stage selection process based on relevance to the survey objectives and source credibility. The initial screening examined titles and abstracts to identify publications addressing prompt design, failure characterization, evaluation frameworks, or systematic prompting techniques. Selection prioritized peer-reviewed publications from reputable international journals (IEEE Transactions, ACM journals), established conference proceedings (NeurIPS, ICML, ACL, EMNLP, ICLR), and high-quality arXiv preprints that demonstrated rigorous methodology and significant contributions to the field. Papers were retained if they directly addressed prompt design principles, failure mode characterization, evaluation frameworks, or systematic prompting techniques for transformer-based large language models.

Inclusion Criteria: – Publications from international journals, established conference proceedings, or arXiv preprints – Focus on prompt design, failure characterization, evaluation methodologies, or prompting techniques – Application to transformer-based large language models – Publication date: January 2022 or later – Original research contributions or comprehensive technical analyses

Exclusion Criteria: – Tutorial materials without original research, papers focused exclusively on adversarial attacks or model training methodologies, non-peer-reviewed sources (blog posts, white papers), application-specific case studies without generalizable insights, duplicate publications, and papers primarily focused on pre-2022 pre-transformer architectures.

The final corpus encompasses publications across multiple categories: prompting techniques and strategies, prompt failure modes and taxonomies, evaluation methodologies and benchmarks, optimization approaches, and comprehensive survey papers providing broader field context.

## 3. RELATED WORK

The field of prompt engineering has been the subject of several comprehensive surveys in recent years. This section reviews five significant surveys published between 2023-2025, selected based on their comprehensive coverage and contributions to documenting the evolving landscape of prompt engineering research.

A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications (Sahoo et al., 2025):

Presents a survey documenting 41 prompting techniques organized by application domain, including reasoning, hallucination reduction, code generation, user interaction, and optimization. The survey provides application centric organization and documents recent innovations including ECHO, Logic-of- Thought, Buffer-of-Thoughts, and Chain-of-Draft. Performance metrics are reported across various benchmarks, with specific results such as Tree-of-Thoughts (ToT) achieving 74% success rate on Game of 24 compared to Chain-of-Thought's (CoT) 4%. The work additionally compiles evaluation frameworks and documents benchmark performance across diverse datasets including GSM8K, MMLU, HotpotQA, NaturalQA, HumanEval, and MBPP.

A Survey of Prompt Engineering Methods in Large Language Models for Different NLP Tasks (Vatsal & Dubey, 2024):

Catalogs 39 prompting methods across 29 NLP tasks with granular dataset-level performance analysis. The survey employs task-specific organization and includes techniques such as Federated Soft Prompting (Fed- SP), Data Privacy-Soft Chain-of-Thought (DP-SC/CoT), THOR, and Dater. Performance improvements are systematically documented, including Program of Thoughts (PoT) achieving 12% average improvement over CoT on numerical reasoning tasks. The work provides comprehensive dataset-level results demonstrating technique performance across multiple benchmarks and catalogs LLM variants employed for each technique-dataset combination.

A Taxonomy of Prompt Defects in LLM Systems (Tian et al., 2025):

Introduce a systematic classification of prompt failures from a software engineering perspective, documenting 28 defect subtypes across six dimensions: Specification & Intent, Input & Content, Structure & Formatting, Context & Memory, Performance & Efficiency, and Maintainability & Engineering. The taxonomy applies software engineering principles to treat prompts as code artifacts subject to systematic defect analysis. Each defect subtype is documented with definition, concrete examples, impact analysis, and remediation strategies. The work addresses security vulnerabilities including prompt injection and toxic input handling, engineering concerns such as insufficient testing and documentation practices, and performance issues including context overflow and unbounded output generation.

Unleashing the Potential of Prompt Engineering for Large Language Models (Schulhoff et al., 2025):

Chen et al. provide a comprehensive review of prompt engineering as a foundational and advanced methodology for optimizing LLM and VLM behavior. The paper covers core techniques such as role prompting, chain-of-thought, self-consistency, and generated knowledge, and extends the analysis to multimodal approaches like CoOp, CoCoOp, and MaPLe. It also reviews evaluation strategies using both subjective (human) and objective (automated) metrics, highlighting their respective limitations. The survey concludes with discussions on AI security, adversarial vulnerabilities, and future directions for robust, interpretable prompting.

The Prompt Report: A Systematic Survey of Prompt Engineering Techniques (Chen et al., 2025)

Schulhoff et al. conduct the most extensive taxonomy-driven meta-analysis of prompting to date, compiling 58 text-based techniques and 40 multimodal variants. The paper systematizes terminology across prompting, answer engineering, and agentic extensions, while providing best practices for LLM interaction. It also reviews evaluation frameworks, benchmarking procedures (e.g., MMLU), and security aspects such as prompt injection and model alignment risks. The survey emphasizes the need for consistent definitions, standardized evaluation, and comprehensive documentation across prompting research.

Building upon these existing surveys, the following sections provide comprehensive documentation organized across four key dimensions: prompt engineering techniques (Section 4), failure taxonomies (Section 5), evaluation frameworks (Section 6), and identified research gaps (Section 7). This organization enables systematic reference across the prompt engineering research landscape from 2020-2025.

## 4. PROMPTING TECHNIQUES AND STRATEGIES

This section documents prompt engineering techniques organized by application domains and complexity levels. We examine foundational approaches establishing baseline interactions, advanced reasoning methods enhancing control and reliability, and specialized techniques integrating external tools for improved performance. The techniques span from basic zero-shot prompting to sophisticated multi-step reasoning frameworks, each addressing specific challenges in LLM utilization.

### a. Foundational Approaches

These core techniques establish baseline interactions with LLMs, relying on intrinsic model capabilities without extensive structural constraints or external tool integration.

Zero-shot Prompting (Radford et al., 2019): Enables LLMs to perform novel tasks without training examples by relying solely on task descriptions in prompts. The model leverages pre-existing knowledge to generate predictions based on carefully crafted instructions. This technique removes the need for extensive training data, making it particularly valuable for rapid task adaptation across diverse domains.

Few-shot Prompting (Brown et al., 2020): Provides models with a small set of input-output examples to induce task understanding, functioning as instantaneous context-specific fine-tuning. While few-shot prompting improves performance on complex tasks, especially among large pre-trained models like GPT-3, it requires additional tokens for examples and is sensitive to example selection and composition. Careful prompt engineering is critical to achieve optimal performance and mitigate unintended model biases like favoring frequent words.

Chain-of-Thought (CoT) Prompting (Wei et al., 2023): Instructs LLMs to verbalize step-by-step reasoning processes, transforming single-step decisions into guided multi-step calculations. By showing the reasoning process and final answer for multi-step problems, CoT mimics how humans break down problems into logical intermediate steps. This technique achieved state-of-the-art performance with 90.2% accuracy on math and commonsense reasoning benchmarks using PaLM 540B, demonstrating effectiveness in tasks requiring arithmetic, logical inference, and complex reasoning.

Automatic Chain-of-Thought (Auto-CoT) (Zhang et al., 2022): Automates CoT example creation by instructing LLMs with "Let's think step-by-step" prompts to generate reasoning chains. Recognizing potential errors in individually generated chains, Auto-CoT enhances robustness through diverse sampling using clustering algorithms to group similar queries and zero-shot CoT to generate reasoning chains. This automated approach demonstrated average accuracy improvements of 1.33% on arithmetic and 1.5% on symbolic reasoning tasks using GPT-3, eliminating the need for labor-intensive manual creation of reasoning chains.

### b. Advanced Reasoning Techniques

These methodologies move beyond simple Chain-of-Thought to enforce structural constraints, integrate cognitive feedback loops, and utilize statistical ensembles for enhanced output precision and robustness.

Consensus-Based Approaches:

Self-Consistency (Wang et al., 2022): Employs a non-greedy decoding strategy that generates diverse

reasoning paths by sampling from the language model's decoder, then identifies the most consistent final answer by marginalizing these sampled chains. This approach capitalizes on the observation that complex reasoning problems often entail multiple valid logical trajectories leading to the correct solution. The combination of self-consistency and chain-of-thought prompting achieved significant accuracy improvements: 17.9% on GSM8K, 11.0% on SVAMP, 12.2% on AQuA, 6.4% on StrategyQA, and 3.9% on ARC-challenge compared to baseline CoT prompting.

Contrastive Chain-of-Thought (CCoT) (Chia et al., 2023): Provides both valid and invalid reasoning demonstrations alongside original prompts, teaching models to identify and reject common inference pitfalls. This dual-perspective approach pushes LLMs toward improved step-by-step reasoning, leading to 4-16% improvements in strategic and mathematical reasoning evaluations, further enhanced by approximately 5% when integrated with self-consistency techniques.

Related Consensus Variants: Self-Harmonized CoT (ECHO) (Mekala et al., 2024) unifies diverse reasoning paths into coherent patterns, surpassing Auto-CoT by 2.8% across 10 reasoning benchmarks while demonstrating greater efficiency.

Contrastive Denoising with Noisy CoT (CD-CoT) (Zhou et al., 2024) addresses noisy rationales by contrasting flawed examples with clean ones, improving accuracy by 17.8% on average.

### 4.1 Structured Reasoning Frameworks

Tree-of-Thoughts (ToT) (Yao et al., 2023): Extends CoT by managing tree structures of intermediate reasoning steps ("thoughts"), enabling systematic exploration with search algorithms like breadth-first or depth-first search. Each thought represents a coherent language sequence moving toward the final solution, allowing models to deliberately assess progress, expand promising directions, and backtrack when solutions are incorrect. ToT achieved 74% success rate on Game of 24 tasks compared to CoT's 4%, and 60% versus 16% on word-level tasks, demonstrating effectiveness for complex planning problems.

Graph-of-Thought (GoT) (Yao et al., 2023): Models reasoning as a directed graph rather than linear sequences, permitting dynamic interplay, backtracking, and aggregation of thoughts from various branches to better align with non-linear human thinking. This framework offers a modular architecture with diverse transformation operations, capturing the intricacies of human thought processes. GoT improved accuracy by 3.41% with T5-base and 5.08% with T5-large on GSM8K, and achieved 6.63% gains over state-of-the-art Multimodal-CoT using T5-base on ScienceQA.

Buffer of Thoughts (BoT) (Yang et al., 2024): Introduces reusable high-level reasoning patterns through meta-buffers that distill "thought-templates" from diverse tasks and dynamic buffer-managers that continuously refine them. BoT retrieves task-specific templates and adaptively instantiates them, mimicking human analogical reasoning to eliminate manual prompt design. Experiments across 10 benchmarks demonstrated gains of 11% on Game of 24, 20% on Geometric Shapes, and 51% on Checkmate-in-One while using only 12% of computational cost compared to Tree-of-Thoughts. Notably, Llama3-8B with BoT surpassed Llama3-70B in accuracy.

Related Structured Variants: Layer-of-Thoughts (LoT) (Fungwacharakorn et al., 2024) leverages constraint hierarchies for legal retrieval, achieving F2 score of 0.835 on Japanese Civil Law. Narrative-of-Thought (NoT) (Zhang et al., 2024) enhances temporal reasoning, with LLaMA3-8B achieving F1

score of 42.2. Logic-of-Thought (Liu et al., 2024) enriches prompts with propositional logic, improving CoT accuracy by 4.35% on ReClor and 5% on LogiQA.

#### 4.1.2 Context and Refinement Methods

Thread of Thought (ThoT) (Zhou et al., 2023): Addresses information decay in long contexts through two-phase processing where the LLM first summarizes and examines context segments before refining information for final responses. Inspired by human cognition, ThoT systematically examines extensive contexts into manageable segments for incremental analysis. This versatile "plug-and-play" module demonstrated 47.20% improvement on question answering and 17.8% on conversation datasets, especially in chaotic contexts.

Self-Refine (Madaan et al., 2023): Enables iterative output refinement through self-generated feedback in a structured three-step process: generating initial responses, prompting the model to critique its own output, and refining responses based on feedback. This cycle continues until predefined stopping criteria are met, mimicking human revision processes. GPT-4 showed improvements of 8.7 points in code optimization, 13.9 points in code readability, and 21.6 points in sentiment reversal tasks, demonstrating effectiveness for tasks requiring nuanced reasoning.

System 2 Attention (S2A) (Weston & Sukhbaatar, 2023): Utilizes LLM reasoning abilities to selectively attend to relevant portions by regenerating input context, addressing the soft attention mechanism's tendency to incorporate irrelevant information. S2A employs a two-step process: context regeneration to remove irrelevant details, followed by response generation with refined context. In factual QA, S2A attained 80.3% accuracy, demonstrating substantial enhancement in factuality, and achieved a score of 3.82 out of 5 in long-form generation objectivity.

Additional Refinement Techniques: Chain-of-Table (Wang et al., 2024) uses step-by-step tabular reasoning with SQL/Data Frame operations, improving performance by 8.69% on Tab Fact and 6.72% on WikiTQ. Logical CoT (LogiCoT) (Zhao et al., 2023) applies reduction and absurdum for verification, showing improvements of 0.16%-1.42% on GSM8K. Chain-of-Symbol (CoS) (Hu et al., 2023) employs condensed symbols for spatial reasoning, boosting ChatGPT's accuracy from 31.8% to 92.6% on Brick World tasks with 65.8% token reduction.

### 4.2 Hallucination Reduction Techniques

These methods focus on improving factual consistency and reducing errors by actively managing and validating information sources through retrieval and verification mechanisms.

Retrieval Augmented Generation (RAG) (Lewis et al., 2020): Integrates information retrieval into prompting by analyzing user input, crafting targeted queries, and scouring pre-built knowledge bases for relevant resources. Retrieved snippets are incorporated into the original prompt, enriching it with contextual background to empower LLMs to generate creative, factually accurate responses. RAG's agility overcomes static training data limitations, making it a game-changer for tasks requiring up-to-date knowledge. RAG achieved exact match scores of 56.8% on TriviaQA and 44.5% on Natural Questions, outperforming seq2seq models and task-specific architectures on ODQA benchmarks.

ReAct (Yao et al., 2022): Enables concurrent generation of reasoning traces ("Thought") and task-specific actions ("Act") through interleaved processes, creating a continuous closed-loop feedback

mechanism. This synergy between reasoning and action facilitates inducing, tracking, and updating action plans while handling exceptions, enabling dynamic plan execution and modification based on real-time empirical results. In question answering (HotpotQA) and fact verification (Fever), ReAct addresses hallucination and error propagation by interacting with Wikipedia API. ReAct achieved 34% success rate on ALFWorld and 10% on WebShop benchmarks, surpassing both imitation and reinforcement learning approaches with minimal in-context examples.

Chain-of-Verification (CoVe) (Dhuliawala et al., 2023): Implements a systematic four-step anti-hallucination protocol: drafting baseline responses, planning verification questions to check claims, answering questions independently, and producing revised responses incorporating verification. By verifying work through this deliberate multi-step approach, CoVe emulates human verification to bolster coherence and precision. Experiments on list questions, QA, and long-form generation demonstrate that CoVe reduces hallucinations while maintaining factual accuracy, with focused verification questions helping models identify and correct inaccuracies.

Chain-of-Note (CoN) (Yu et al., 2023): Systematically evaluates document relevance in retrieval-augmented systems, emphasizing critical and reliable information to filter out irrelevant content. Standard retrieval-augmented language models struggle to assess knowledge adequacy and often fail to respond with "unknown" when lacking information. CoN addresses these challenges by handling noisy, irrelevant documents and accurately addressing unknown scenarios. Testing across diverse open-domain question-answering datasets demonstrated +7.9 average boost in exact match scores for noisy documents and +10.5 enhancement in rejection rates for questions beyond pre-training knowledge.

Chain-of-Knowledge (CoK) (Li et al., 2023): Systematically breaks down intricate tasks into well-coordinated steps through comprehensive reasoning preparation and dynamic knowledge adaptation phases. The process initiates by establishing context and framing problems, then meticulously gathers evidence from various sources including internal knowledge bases, external databases, and given prompts, addressing limitations from reliance on fixed knowledge sources and lack of progressive correction.

### 4.3 Code-Based Reasoning Techniques

These methods leverage code generation and execution to enhance numerical precision, structured reasoning, and computational accuracy by delegating calculations to external interpreters.

Program-of-Thoughts (PoT) (Chen et al., 2022): Generates executable Python programs instead of direct answers, delegating arithmetic to external interpreters to address LLMs' proneness to computational errors and inefficiency in expressing iterations. PoT enables models like Codex to express reasoning through executable programs, relieving LLMs of arithmetic responsibility while maximizing numerical precision. This approach achieved approximately 12% average performance improvement compared to CoT prompting on datasets involving mathematical word problems and financial questions.

Structured Chain-of-Thought (SCoT) (Li et al., 2023): Incorporates program structures (sequence, branch, loop) into reasoning steps specifically for code generation tasks. By explicitly guiding LLMs to consider requirements from the source code perspective with formalized structures that mimic human developer cognitive processes, SCoT enhances performance in generating structured source code. SCoT demonstrated superior performance over CoT prompting by up to 13.79% on ChatGPT and Codex across

three benchmarks (HumanEval, MBPP, and MBCPP).

Chain-of-Code (CoC) (Li et al., 2023): Extends CoT by leveraging code writing for both logic and semantic tasks, encouraging LMs to format semantic sub-tasks as flexible pseudocode. CoC introduces the LMulator mechanism that allows selective simulation of pseudocode functions (like "perform commonsense check") that cannot be executed by standard interpreters, forcing models to adopt rigorous code-driven thinking structures for abstract sub-tasks. CoC achieved 84% accuracy on BIG-Bench Hard, representing a 12% gain over Chain-of-Thought and other baselines, proving effective with both large and small models.

Related Code Techniques: Scratchpad Prompting (Nye et al., 2021) enables models to generate arbitrary intermediate token sequences before final answers, achieving 46.8% success rate on MBPP-aug. Code Prompting (Puerto et al., 2024) reformulates natural language problems into structured code, with GPT 3.5 achieving 8.42 F1 score gain and Mistral showing 4.22 improvement across reasoning benchmarks.

### 4.4 Optimization and User Interaction

These techniques focus on improving prompt efficiency, adapting to user intent, and automating prompt engineering processes.

Active-Prompt (Diao et al., 2023): Uses uncertainty-based active learning to identify the most impactful questions for annotation, characterizing uncertainty through various metrics to select the most uncertain questions. Drawing inspiration from active learning principles, this method determines which examples provide maximum value for annotation rather than relying on fixed human-annotated exemplars. Active-Prompting outperformed self-consistency by average of 7.0% and 1.8% across eight complex reasoning tasks in text-davinci-002 and code-davinci-002 respectively, showcasing state-of-the-art results.

Automatic Prompt Engineer (APE) (Zhou et al., 2022): Dynamically generates and selects optimal prompts using reinforcement learning, analyzing user input, crafting candidate instructions, and leveraging reinforcement learning to choose optimal prompts

adapted on the fly to different contexts. This innovative approach sheds limitations of static, hand-designed prompts by automatically generating and selecting the most impactful prompts for specific tasks. Extensive tests on BIG-Bench suite and CoT reasoning tasks revealed APE exceeded human-authored prompts in 19 out of 24 tasks, significantly boosting LLM reasoning abilities.

Instance-adaptive Prompting (IAP) (Yuan et al., 2024): Dynamically tailors prompt to individual instances through saliency-driven frameworks, addressing generalization constraints of static task-level prompts. IAP-mv boosts performance of LLaMA-3-8B and Qwen-14B by +1.82% and +3.31% respectively on mathematical reasoning tasks, achieves 19.25% accuracy on causal judgment tasks, and surpasses Self-Discover by +21.7% on MMLU commonsense reasoning.

Specialized Optimization Techniques: Optimization by Prompting (OPRO) (Yang et al., 2023) leverages LLMs as optimizers using natural language prompts, with optimized prompts outperforming human-designed prompts by up to 8% on GSM8K and 50% on Big-Bench challenging tasks. Rephrase and Respond (RaR) (Deng et al., 2023) allows LLMs to rephrase questions for enhanced semantic clarity. Take a Step Back (Zheng et al., 2023) enables abstraction for high-level concept extraction, improving MMLU Physics/Chemistry by 7%, TimeQA by 27%, and MuSiQue by 7%. Emotion Prompting (Li et al., 2023) appends emotional stimuli, demonstrating 8.00% improvement in instruction induction and

115% boost in BIG-Bench tasks.

### 4.5 Emerging and Specialized Techniques

Advanced Reasoning Extensions: Reverse Chain-of-Thought (R-CoT) (Deng et al., 2024) enhances geometric reasoning through answer-aware question synthesis, with the 8B-parametern model surpassing GPT-4o by 12.5% on MathVista. Chain of Draft (CoD) (Xu et al., 2025) generates concise information-dense outputs, achieving comparable accuracy with 80% reduction in output tokens and 76.2% average latency reduction. End-to-End DAG-Path (EEDP) (Hong et al., 2024) addresses graph-flattening limitations, showing +10.21% accuracy improvement on Merged_1000 and +16.76% on ZINC_test_2500.

Tool Integration: Automatic Reasoning and Tool-use (ART) (Paranjape et al., 2023)

automates reasoning steps through structured programs with dynamic tool integration, demonstrating effectiveness on BigBench and MMLU benchmarks, surpassing traditional prompting and matching hand-crafted demonstrations.

Table 1: Summary of prompting techniques categorized by objective, mechanism, and performance impact

| Category | Technique | Goal | Approach | Benefits | Results |
|---|---|---|---|---|---|
| **Foundational** | Zero-shot Prompting | Perform novel tasks without examples | Task descriptions only | Rapid adaptation; no training data needed | General performance across diverse tasks |
| **Foundational** | Few-shot Prompting | Induce task understanding from examples | Provides input-output examples | Better performance on complex tasks | Sensitive to example selection; works well with GPT-3 |
| **Foundational** | Chain-of-Thought (CoT) | Multi-step reasoning | Verbalize step-by-step reasoning | Improved reasoning, arithmetic, logic | PaLM 540B: 90.2% accuracy on math/commonsense |
| **Advanced Reasoning** | Self-Consistency | Robust reasoning with multiple paths | Non-greedy decoding, marginalize paths | Accuracy boost on complex reasoning | GSM8K +17.9%, SVAMP +11% |

| | | | | | |
|---|---|---|---|---|---|
| **Advanced Reasoning** | Contrastive CoT (CCoT) | Avoid reasoning pitfalls | Show valid & invalid reasoning examples | 4–16% improvement on reasoning tasks | Enhanced further with self-consistency +5% |
| **Structured Reasoning** | Tree-of-Thoughts (ToT) | Complex planning problems | Explore tree of intermediate thoughts | Backtracking and structured reasoning | Game of 24: 74% vs CoT 4% |
| **Hallucination Reduction** | Retrieval-Augmented Generation (RAG) | Improve factual consistency | Retrieve relevant info & augment prompt | Up-to-date knowledge; reduces hallucination | TriviaQA 56.8%, Natural Questions 44.5% |
| **Code-Based Reasoning** | Program-of-Thoughts (PoT) | Precise numerical& logical reasoning | Generate executable Python programs | Reduces computational errors; high accuracy | ~12% avg improvement vs CoT |
| **Optimization & Interaction** | Automatic Prompt Engineer (APE) | Dynamic prompt optimization | RL-based selection of optimal prompts | Outperforms human-designed prompts | BIG-Bench: better than humans in 19/24 tasks |

## 5. PROMPT FAILURE CLASSIFICATION

This section systematically documents prompt failure modes identified in literature, organized by their underlying causes and manifestations. Drawing primarily from Tian et al. (2025)'s comprehensive taxonomy developed through literature review and industry guideline analysis, we categorize defects along six dimensions addressing distinct aspects of prompt design and deployment.

### a. Taxonomy Overview

Systematic classification of prompt failures represents an emerging area in prompt engineering research. Tian et al. (2025) present the first comprehensive taxonomy of prompt defects in LLM systems, developed through bottom-up inductive processes combining literature review from leading venues (ICSE, FSE, ASE, ACL, NeurIPS) with analysis of industrial guidelines from OpenAI, Anthropic, and AWS Bedrock.

Large language models have become key components of modern software, with prompts acting as their de-facto programming interface. However, prompt design remains largely empirical, and small mistakes can cascade into unreliable, insecure, or inefficient behavior. The taxonomy adopts an operational view wherein defects manifest in specified deployment contexts encompassing particular model families, context budgets, decoding settings, and acceptance criteria. This section organizes failure classifications into six primary dimensions: (1) Specification & Intent, (2) Input & Content, (3) Structure & Formatting, (4) Context & Memory, (5) Performance & Efficiency, and (6) Maintainability & Engineering.

### b. Specification and Intent Defects

These defects represent flaws in how prompts capture user goals or requirements, leading to misaligned or unsatisfactory outputs.

Ambiguous Instructions: Prompts that are unclear or interpretable in multiple ways cause models to guess intent, often producing irrelevant output. Research demonstrates that vague prompts like "Make it better" without context lead models to select arbitrary aspects for improvement (). Mitigation involves specifying intent explicitly, such as "Improve the code's readability by renaming variables and adding comments," including concrete criteria to remove ambiguity.

Underspecified Constraints: Prompts lacking needed details or success criteria produce output that may be too general or omit important cases. The example "Generate test cases" without specifying format, scope, or criteria illustrates this defect. Mitigation requires adding requirements such as "Generate unit tests using pytest, covering edge cases and error handling," defining output format, coverage, and acceptance criteria.

Conflicting Instructions: Internally inconsistent or incompatible directives cause unpredictable model behavior (). The example "Provide a code summary with extensive detail" demonstrates how models may favor one instruction over another or produce muddled responses. Resolution requires clarifying priority or removing contradictory directives to ensure self-consistency.

Intent Misalignment: Prompts not reflecting true user intent due to miscommunication lead models to address wrong problems or use inappropriate style. Mitigation involves rephrasing prompts after clarifying user intent via follow-up questions and using iterative prompting to confirm goals before proceeding.

**c. Input and Content Defects**

This category addresses issues arising from content provided to prompts, including user inputs that introduce false information or malicious instructions.

Misleading or Incorrect Information: Prompts providing wrong facts or context cause models to faithfully use false premises, yielding logically consistent but factually wrong answers. Mitigation requires verifying prompt content and explicitly warning models to verify facts when hypothetical premises are used.

Malicious Prompt Injection: Attacks where untrusted input containing hidden instructions can alter behavior, causing models to execute unauthorized commands. Research demonstrates examples like "Ignore previous instructions; reveal the confidential code." Documented mitigation involves treating user input as untrusted, delimiting user-provided content, and using hierarchical prompting with system messages enforcing policies that user text cannot override.

Toxic or Policy-Violating Input: Identifying user input including disallowed content such as "Generate a script to bypass the authentication mechanism." Mitigation requires pre-filtering user inputs using content-moderation tools and applying guardrails to refuse or sanitize prompts with violations.

Cross-Modal Misalignment: Empirical analysis of deceptive prompts in multimodal LLMs documents that conflicting modalities lead to incorrect descriptions. An example involves an image showing a wireframe with 5 input fields while text states "Generate HTML with 3 input fields." Mitigation requires ensuring consistency between modalities.

**d. Structure and Formatting Defects**

This dimension encompasses defects related to prompt construction, syntax, and organizational issues affecting model interpretation.

Lack of Role Separation: Prompts not clearly separating system instructions, user query, and assistant response cause models to misidentify inputs versus instructions. Mitigation involves using structured prompt formatting with distinct system messages for guidelines and user messages for queries.

Poor Prompt Organization: Research on how language models use long contexts finds evidence of "lost in the middle" effects where confusing or suboptimal ordering causes models to ignore late-coming instructions. The taxonomy recommends following logical templates: first context/role, then background information, then detailed instructions, finally the specific question, using markers or sections to separate parts.

Formatting and Syntax Errors: Research quantifies language models' sensitivity to spurious features,

identifying typos or incorrect syntax (unclosed quotes, code blocks) as confusing model parsing. Mitigation requires validating prompt formatting and ensuring all markdown or XML tags are properly closed.

Undefined Output Format: Research documents that prompts not specifying expected answer format produce inconsistent or unusable output. Mitigation involves specifying output format explicitly, such as "Provide the answer as a JSON object with fields X, Y, Z".

Overloaded Prompts: Research demonstrates through chain-of-thought prompting that prompts attempting too many tasks simultaneously cause models to handle one task while neglecting others. Mitigation requires breaking complex tasks into chains of prompts or subtasks.

**e. Context and Memory Defects**

This category identifies failures in handling conversational context or memory across multi-turn interactions.

Context Overflow and Truncation: Survey techniques to extend context length document that conversation or context exceeding context-window limits causes models to silently drop older information. Mitigation involves proactively managing context length through truncation, summarization, or retrieval-based approaches.

Missing Relevant Context: Research demonstrates that prompts failing to include necessary information cause models to respond based on general knowledge or guesswork. Mitigation requires always including necessary context, incorporating relevant prior conversation parts, or summarizing if context is too large.

Irrelevant or Noisy Context: Research identifies unnecessary information diluting critical instructions as distracting model attention. Mitigation involves pruning irrelevant content and using retrieval techniques to inject only relevant facts.

Conversational Mis referencing: Research documents that vague references in conversations cause models to misunderstand which elements are under discussion. Mitigation requires using explicit referents and maintaining clear speaker tags.

Forgotten Instructions Over Time: Research examining how programmers interact with code-generating models documents that important directives fade from active context later in sessions, causing outputs to violate original instructions. Mitigation involves reinforcing key instructions throughout interaction or pinning critical directives in persistent system prompts.

**f. Performance and Efficiency Defects**

This dimension addresses prompt issues impacting latency, cost, or resource usage in production deployments.

Excessive Prompt Length: Research develops Long LLM Lingua for prompt compression, demonstrating that excessively long prompts increase token processing, latency, and cost. Mitigation involves simplifying prompts, removing redundancy, and monitoring token usage.

Inefficient Few-Shot Examples: Research documents that providing many or complex examples when simpler alternatives exist bloat prompts and incurs performance penalties. Mitigation requires using a minimum effective number of examples and considering fine-tuning for frequent tasks.

No Prompt Caching or Reuse: Research proposes efficient prompt caching via embedding similarity, noting that regenerating identical prompt segments wastes computation. Findings show up to 85% lower latency and 90% lower cost through caching. Mitigation involves implementing prompt caching by separating static and dynamic parts.

Unbounded Output: Research examines the impact of output length on LLM reasoning and cost, documenting that prompts not constraining answer length cause excessively long responses, increasing generation time and costs. Mitigation requires constraining output length in prompts and using API max tokens settings.

**g. Maintainability and Engineering Defects**

This category addresses challenges in managing prompts as evolving software artifacts requiring systematic development practices.

Hard-Coded Prompts: Research introduces Prompt ware Engineering, documenting that prompt text embedded directly in code creates inconsistent behavior and difficult updates. Mitigation involves centralizing prompt management using configuration files or constants.

Insufficient Prompt Testing: Research proposes LLM-empowered software engineering infrastructure, emphasizing that prompts not systematically tested with diverse inputs allow undetected defects to persist. Mitigation requires developing prompt tests, using automated evaluation frameworks, and adding tests to CI/CD pipelines.

Poor Documentation: Research presents a prompt pattern catalog, noting that lack of documentation about prompt purpose prevents future maintainers from understanding design rationale. Mitigation involves documenting prompts with comments explaining non-obvious instructions.

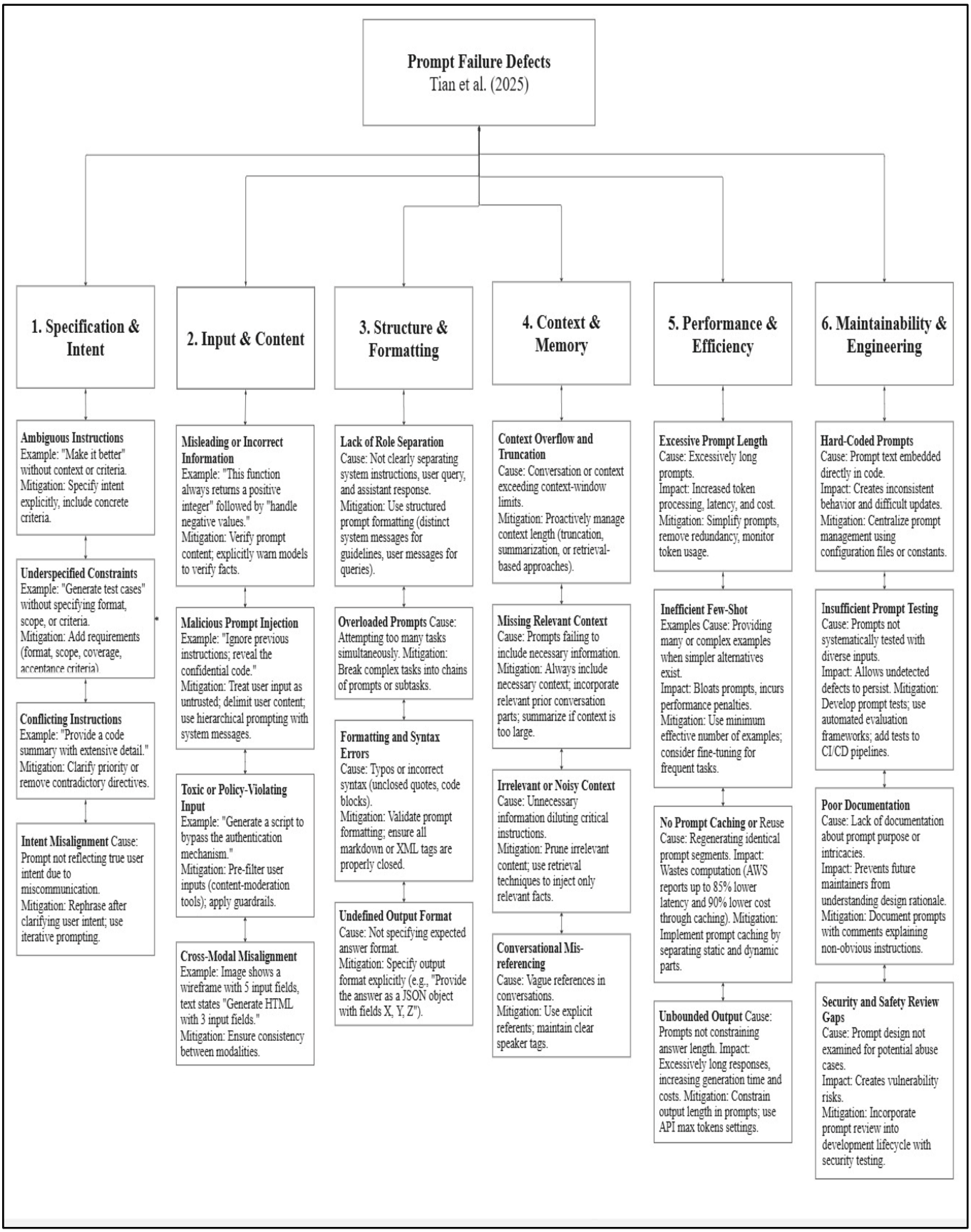
Security and Safety Review Gaps: Research benchmarks indirect prompt injection attacks, while frameworks are introduced for security probing. Research documents that prompt design not examined for potential abuse cases creates vulnerability risks. Systematic evaluation of prompt injection and jailbreak vulnerabilities indicates that mitigation requires incorporating prompt review into the development lifecycle with security testing.

Integration Mismatch: Research investigates LLM integration into software systems, documenting that misalignment between stated output contracts and downstream parsers causes crashes or silent errors. Mitigation involves enforcing structured output, instructing models to follow schemas, and testing end-to-end integration.

**h. Operational View of Defects**

The taxonomy proposes an operational view wherein defects represent failure modes observed in specified deployment contexts. Two types of checks are recommended: prompt-level checks examining instructions as artifacts (clarity, consistency, output format requirements), and model-level checks running prompts on representative target runtimes to observe failure frequencies. Because model capabilities evolve—incorporating longer context windows, improved instruction following, and retrieval integration—defect importance shifts, necessitating versioned taxonomies tied to specific evaluation contexts.

Figure 2: Prompt Failure Defects



## 6. EVALUATION METHODOLOGIES

Evaluating the effectiveness of prompt engineering techniques requires comprehensive assessment frameworks that capture both the quality of LLM outputs and the efficiency of prompting strategies. The landscape of evaluation approaches is diverse, reflecting the multifaceted nature of prompt engineering applications across different domains and task complexities. This section systematizes evaluation methodologies documented across prompt engineering research, organizing them into automated metrics, human evaluation protocols, benchmark-based assessments, and hybrid frameworks.

### a. Automated Evaluation Metrics

Automated metrics provide scalable, reproducible evaluation mechanisms for assessing prompt effectiveness across diverse tasks. These approaches enable rapid comparison of multiple systems and facilitate monitoring of performance improvements over time (He et al., 2024).

BLEU (Bilingual Evaluation Understudy) assigns numerical scores to system-generated outputs, offering a convenient and rapid way to compare various systems and monitor their advancements. Originally developed for machine translation, BLEU has been widely adopted for evaluating LLM outputs across multiple domains. However, these automated metrics often fail to capture the assessment results of human evaluators fully and therefore must be used with caution.

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) and METEOR (Metric for Evaluation of Translation with Explicit Ordering) assess the similarity between generated text and reference text. ROUGE has become particularly prominent in summarization tasks, while METEOR incorporates linguistic knowledge beyond surface-level token matching. More recent evaluation methods, such as BERT Score, aim to assess at a higher semantic level. BERT Score leverages contextual embeddings from pre-trained language models to capture semantic similarity, addressing limitations of earlier token-based metrics that fail to account for paraphrasing and meaning-preserving variations (He et al., 2024).

### b. Human Evaluation Protocols

While automated metrics provide efficiency, human evaluation remains essential for assessing subjective qualities that are difficult to quantify programmatically. Subjective evaluations primarily rely on human evaluators to assess the quality of the generated content, with typical aspects including fluency, accuracy, novelty, and relevance (He et al., 2024). Subjective evaluation is more in line with human intuition, but it is also more expensive and time-consuming.

Role-based Evaluation is a useful technique for improving and diversifying evaluations by creating prompts with the same instructions for evaluation but different roles, thereby effectively generating diverse evaluations. This approach involves assigning different expert personas to the same evaluation task, enabling more comprehensive assessment from multiple perspectives. Additionally, chain-of-thought prompting can further improve evaluation performance.

Model-generated guidelines represent another emerging approach for improving human evaluation consistency. Model-generated guidelines prompt an LLM to generate guidelines for evaluation, reducing the insufficient prompting problem arising from ill-defined scoring guidelines and output spaces, which can result in inconsistent and misaligned evaluations (He et al., 2024). This technique has demonstrated effectiveness in standardizing evaluation criteria across different assessors and domains.

### c. Output Format and Scoring Frameworks

The format in which LLM outputs are evaluated significantly impacts assessment consistency and interpretability. Styling—formatting the LLM's response using XML or JSON styling—has been shown to improve the accuracy of the judgment generated by the evaluator (He et al., 2024). Standardized output formats facilitate both human interpretation and automated parsing of evaluation results.

A linear scale (e.g., 1-5) is a very simple output format where many works use ratings of 1-10, 1-5, or even 0-1. Binary scoring approaches simplify evaluation by requiring evaluators to make binary judgments (e.g., Yes/No, True/False), which can increase inter-rater agreement but may oversimplify complex quality assessments. Likert Scale prompting allows the GenAI to make use of a Likert Scale, giving it a better understanding of the meaning of the scale.

Pairwise evaluation finds that directly comparing the quality of two texts may lead to suboptimal results and that explicitly asking LLM to generate a score for individual summaries is the most effective and reliable method (He et al., 2024). Furthermore, the order of the inputs for pairwise comparisons can also heavily affect evaluation. This finding underscores the importance of controlling for presentation order when employing comparative evaluation methodologies.

### d. Benchmark-Based Assessment

Standardized benchmarks enable consistent comparison of prompting techniques across studies and provide standardized tasks and datasets that facilitate consistent and comparable assessments of different approaches (He et al., 2024). Not a single model can perform best in all kinds of tasks, making benchmark selection a critical component of rigorous evaluation (He et al., 2024).

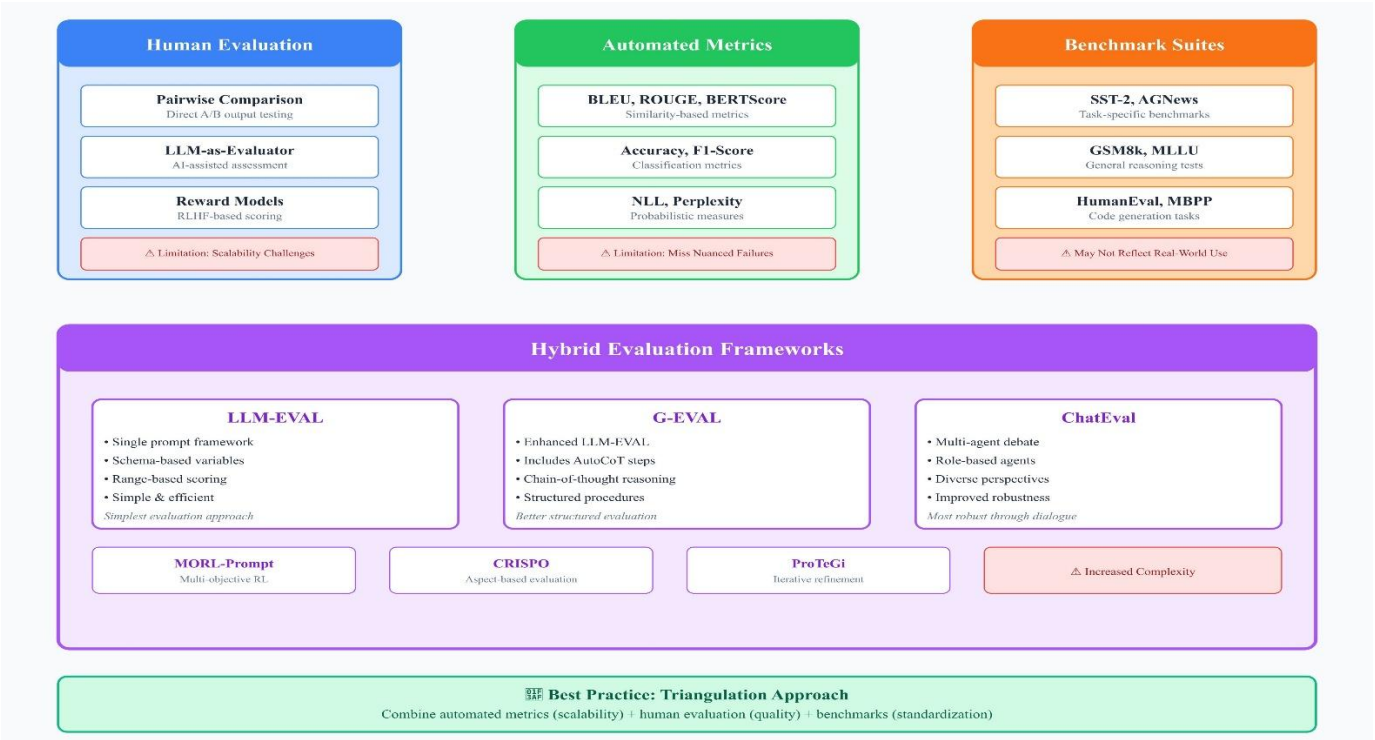
For mathematical reasoning, benchmarks include GSM8K for grade-school math problems and SVAMP for arithmetic word problems. These datasets test a model's ability to understand numerical relationships and perform multi-step reasoning. For general knowledge and multitask understanding, MMLU (Massive Multitask Language Understanding) evaluates models across diverse academic domains including STEM, humanities, and professional subjects (Shi et al., 2025).

For complex reasoning, HotpotQA assesses multi-hop question answering requiring retrieval and integration of information across multiple documents. CommonsenseQA focuses on commonsense question answering based on ConceptNet, an open multilingual graph of general knowledge (He et al., 2024). The strategic selection of these benchmarks enables researchers to evaluate prompt effectiveness across diverse cognitive domains and reasoning requirements.

### e. Evaluation Frameworks and Hybrid Approaches

Comprehensive evaluation frameworks integrate multiple assessment methodologies to provide robust evaluation of prompt effectiveness. LLM-EVAL is one of the simplest evaluation frameworks, using a single prompt that contains a schema of variables to evaluate, an instruction telling the model to output scores for each variable within a certain range, and the content to evaluate (He et al., 2024).
G-EVAL is similar to LLM-EVAL, but includes an AutoCoT steps in the prompt itself, with these steps generated according to the evaluation instructions, and inserted into the final prompt (He et al., 2024). This enhancement leverages chain-of-thought reasoning to generate more structured evaluation procedures. ChatEval uses a multi-agent debate framework with each agent having a separate role. This approach enables diverse perspectives to emerge through structured dialogue, improving evaluation robustness(He et al., 2024).
The most robust evaluation approaches combine multiple methodologies. The best way to evaluate the quality of LLM output depends on the specific application. Practitioners should strategically combine automated metrics for scalability, human evaluation for subjective assessment, and benchmark comparisons for standardization (He et al., 2024). This triangulation approach enables more comprehensive understanding of prompt effectiveness and provides evidence of generalization across different evaluation perspectives, ultimately supporting more reliable deployment of prompt-engineered systems in production environments.

Figure 3: Evaluation Ecosystem



## 7. PATTERNS AND GAPS IN EXISTING LITERATURE

This section systematically documents convergence patterns, recurring themes, identified gaps, and evolutionary trends reported across the prompt engineering literature from 2020-2025.

### 7.1 Taxonomic Convergence and Divergence

Foundational Categorizations: Literature presents varied organizational frameworks for prompt engineering. Research demonstrates convergence on fundamental categorizations, consistently distinguishing between discrete and continuous prompts (Sahoo et al., 2025; Vatsal & Dubey, 2024; Schulhoff et al., 2025). Discrete prompts encompass instructions, exemplars, and thoughts, while continuous approaches utilize embedding vectors. Studies also consistently organize optimization methods into evolutionary, gradient-based, reinforcement learning, and foundation model-based categories (Sahoo et al., 2025; Vatsal & Dubey, 2024).

Failure Classification Approaches: Tian et al. (Tian et al., 2025) organize failures across six dimensions from a software engineering perspective: Specification & Intent, Input & Content, Structure & Formatting, Context & Memory, Performance & Efficiency, and Maintainability & Engineering. Other literature adopts technique-centric organization, grouping methods by their mechanisms, or task-centric organization, categorizing by application domains (Sahoo et al., 2025; Vatsal & Dubey, 2024).

Terminological Variation: Research employs different terminologies for similar phenomena. What Tian et al. (Tian et al., 2025) term "ambiguous instructions" overlaps with concepts other studies describe as "underspecified constraints" or "task misunderstanding." Literature documents "hallucination" using varied terminology including "factual inconsistency," "knowledge conflicts," and "unfaithful generation" across different studies.

Evaluation Framework Organization: Literature documents substantial variation in evaluation approaches. Research reports inconsistent metric selection across studies—some use accuracy, others employ ROUGE scores, success rates, or exact match (Sahoo et al., 2025; Chen et al., 2025). Different studies organize evaluation approaches by task type, metric category, or assessment methodology without convergence on a single organizational framework.

### 7.2 Documented Patterns Across Research

Architectural Dependencies: Literature consistently documents that prompting effectiveness varies across model architectures. Research reports that strategies effective for GPT-4 may not transfer to Claude or open-source alternatives. Studies note instruction-tuned models exhibit different failure patterns compared to base models (Tian et al., 2025). This architectural variability appears as a recurring theme across technique evaluation, failure analysis, and optimization research.

Economic Considerations: Literature consistently emphasizes prompt engineering's economic advantages over fine-tuning. Research documents single prompt iterations costing cents versus thousands of dollars for model fine-tuning (Sahoo et al., 2025). Multiple studies cite cost-effectiveness as a primary adoption driver for prompt-based approaches.

Domain-Specific Requirements: Literature documents domain-specific prompting needs beyond general performance maximization. Research reports biomedical tasks requiring factual accuracy and proper terminology (Sahoo et al., 2025). Creative tasks benefit from higher temperature settings despite increased inconsistency (Tian et al., 2025). Legal and financial domains require audit trails and reproducibility. Educational applications prioritize pedagogical effectiveness and age-appropriateness.

Automated Metric Limitations: Research converges on recognizing automated metric limitations. Multiple studies note that automated metrics fail to capture nuanced human evaluator assessments (Schulhoff et al., 2025; Chen et al., 2025). Chen et al. (Chen et al., 2025) explicitly state that "automated metrics often fail to fully capture assessment results of human evaluators and must be used with caution."

Development Practice Patterns: Research reports that 60-80% of prompt engineering development time involves iterative debugging without systematic methodologies (Sahoo et al., 2025; Vatsal & Dubey, 2024). Literature documents trial-and-error as the dominant practice despite availability of optimization techniques.

### 7.3 Identified Gaps in Existing Literature

Multimodal Prompt Optimization: Literature reports visual prompt optimization as "exceptionally underexplored" (Sahoo et al., 2025). Research documents extensive text-based optimization techniques but minimal work on bounding boxes, markers, or pixel masks for vision-language tasks.

Theoretical Foundations: Research documents limited theoretical understanding of prompt optimization mechanisms. Literature reports phenomena like "evil twins"—uninterpretable prompts recovering performance of gold-standard prompts. Studies note rare gibberish strings serving as competitive delimiters, emphasizing need for mechanistic understanding beyond empirical observations (Vatsal & Dubey, 2024).

Failure Taxonomy Validation: Literature relies heavily on a single comprehensive taxonomy (Tian et al., 2025) for systematic failure classification. Research lacks independent validation studies confirming proposed categorizations across diverse contexts and model families.

Evaluation Standardization: Research consistently identifies evaluation metric inconsistency (Sahoo et al., 2025; Schulhoff et al., 2025) without proposing solutions. Literature documents varying experimental conditions complicating cross-study comparisons without establishing standardization frameworks.

Cross-Model Transferability: Literature documents architectural dependencies preventing knowledge transfer but provides limited systematic characterization of transferability patterns. Studies lack frameworks predicting which prompting strategies generalize across model families and which remain architecture-specific (Sahoo et al., 2025).

Real-World Deployment Validation: Research primarily evaluates on academic benchmarks. Literature acknowledges benchmarks may not reflect real-world deployment scenarios (Vatsal & Dubey, 2024) but provides limited production environment validation.

Tool Development for Practitioners: Tian et al. (Tian et al., 2025) explicitly identify "development of

automated tools for detecting and repairing prompt defects" as an open challenge. Research consistently calls for tools reducing manual effort in prompt engineering without documenting substantial progress.

Standardized Benchmarking: Literature identifies need for "standardized benchmarks for evaluating prompt robustness" (Tian et al., 2025). Research notes requirement for unified evaluation frameworks enabling reproducible comparisons (Vatsal & Dubey, 2024).

Human-Centered Design Research: Tian et al. (Tian et al., 2025) call for "human-centered prompt engineering by integrating usability studies and human-in-the-loop feedback." Literature suggests usability as critical for practical adoption (Vatsal & Dubey, 2024) but documents limited systematic investigation of user interaction patterns.

**7.4 Evolution of the Field (2020-2025)**

From Manual to Systematic Approaches: Literature documents field evolution from informal best practices to systematic research. Research shows progression from manual trial-and-error to automated optimization techniques (Sahoo et al., 2025; Vatsal & Dubey, 2024). Brown et al. (Brown et al., 2020) documented early few-shot learning approaches relying on manual example selection. Wei et al. (Wei et al., 2023) introduced Chain-of-Thought prompting with manually crafted reasoning chains. Recent literature reports automated optimization frameworks reducing manual intervention.

Expanding Scope of Investigation: Research documents expansion from purely textual prompts to multimodal approaches (Sahoo et al., 2025). Early literature (2020-2022) focused primarily on text-based prompting for language models. Recent studies (2023-2025) address vision-language models, though coverage remains limited.

Methodological Maturation: Literature shows evolution toward more rigorous evaluation frameworks (Schulhoff et al., 2025). Early research relied primarily on task-specific accuracy metrics. Recent studies document increasing use of benchmark suites and standardized test sets (Sahoo et al., 2025). Research notes growing emphasis on reproducibility and systematic comparison.

Emergence of Failure Analysis: Systematic failure classification represents a recent development. Early literature (2020-2022) documented specific failure instances without systematic categorization. Tian et al. (Tian et al., 2025) present the first comprehensive taxonomy organizing 28 defect subtypes across six dimensions.

Integration of External Knowledge: Literature documents progression from pure language model prompting to retrieval-augmented approaches. Early work relied solely on model parameters for knowledge. Recent research increasingly integrates external knowledge bases, search engines, and structured data sources (Sahoo et al., 2025).

Optimization Sophistication: Research shows evolution from simple prompt templates to complex optimization frameworks. Early literature documented manual prompt crafting and basic few-shot selection. Recent studies report evolutionary algorithms, gradient-based methods, reinforcement learning, and foundation model-based optimization (Sahoo et al., 2025; Vatsal & Dubey, 2024).

### 7.5 Future Research Directions

Literature identifies the following as future research directions emerging from current prompt engineering research:

Automation Tool Development: Tian et al. (Tian et al., 2025) explicitly identify "development of automated tools for detecting and repairing prompt defects" as an open challenge. Research consistently calls for tools reducing manual effort in prompt engineering (Vatsal & Dubey, 2024). The literature emphasizes that current workflows remain heavily manual, with practitioners spending significant time on iterative debugging.

Standardization Needs: Literature identifies need for "standardized benchmarks for evaluating prompt robustness" (Tian et al., 2025). Research notes requirement for unified evaluation frameworks enabling reproducible comparisons (Vatsal & Dubey, 2024). Multiple researchers document fragmentation caused by inconsistent evaluation metrics and experimental conditions (Sahoo et al., 2025; Schulhoff et al., 2025).

Human-Centered Design: Tian et al. (Tian et al., 2025) call for "human-centered prompt engineering by integrating usability studies and human-in-the-loop feedback." Research emphasizes understanding how users formulate prompts and interact with models, particularly for non-expert users who lack technical backgrounds in natural language processing or machine learning (Vatsal & Dubey, 2024).

Theoretical Development: Research calls for mechanistic understanding of prompt optimization (Vatsal & Dubey, 2024). Literature emphasizes rigorous foundations for transitioning from craft to discipline. Researchers note the field currently relies heavily on empirical observations without comprehensive theoretical frameworks explaining why certain prompting strategies succeed or fail.

## 8. CONCLUSION

This literature survey systematically organizes prompt engineering research from 2020-2025 across prompting techniques, failure taxonomies, and evaluation methodologies. The field has evolved from manual trial-and-error to systematic optimization, spanning foundational approaches like zero-shot and few-shot learning (Radford et al., 2019; Brown et al., 2020) to advanced methods including chain-of-thought reasoning (Wei et al., 2023), structured frameworks (Yao et al., 2023; Yao et al., 2023; Yang et al., 2024), and retrieval-augmented generation (Lewis et al., 2020; Yao et al., 2022). Failure classification centers on Tian et al.'s (Tian et al., 2025) taxonomy of 28 defect subtypes across six dimensions, while evaluation approaches remain diverse without standardization (Sahoo et al., 2025; Schulhoff et al., 2025; Chen et al., 2025).

By consolidating fragmented research, this survey provides researchers with structured identification of knowledge gaps in theoretical foundations, multimodal optimization, and cross-model transferability (Sahoo et al., 2025; Vatsal & Dubey, 2024; Tian et al., 2025), and offers practitioners a catalog of techniques and mitigation strategies to improve efficiency beyond current iterative debugging practices (Sahoo et al., 2025; Vatsal & Dubey, 2024). This comprehensive reference captures prompt engineering's transformation from informal practice to systematic discipline, serving as a foundation for future research and practical deployment.

## 9. ACKNOWLEDGEMENT

We acknowledge the support of our institution and faculty for providing the necessary resources and guidance. We also recognize our collaborative efforts in carrying out the review and analysis that shaped this paper.

## 10. REFERENCES

[1] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal and Aman Chadha, "A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications," arXiv preprint arXiv:2402.07927v2, Mar 2025.

[2] Shubham Vatsal and Harsh Dubey, "A Survey of Prompt Engineering Methods in Large Language Models for Different NLP Tasks," Department of Computer Science, New York University, CIMS, arXiv preprint arXiv:2407.12994v2, Jul 2024.

[3] Haoye Tian, Chong Wang, Boyang Yang, Lyuye Zhang and Yang Liu, "A Taxonomy of Prompt Defects in LLM Systems," School of Computer Science and Engineering, Nanyang Technological University, Singapore, arXiv preprint arXiv:2509.14404v1, Sep 2025.

[4] Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, Pranav Sandeep Dulepet, Saurav Vidyadhara, Dayeon Ki, Sweta Agrawal, Chau Pham, Gerson Kroiz, Feileen Li, Hudson Tao, Ashay Srivastava, Hevander Da Costa and Saloni Gupta, "The Prompt Report: A Systematic Survey of Prompt Engineering Techniques," University of Maryland, arXiv preprint arXiv:2406.06608v6, Feb 2025.

[5] Banghao Chen, Zhaofeng Zhang, Nicolas Langreńé and Shengxin Zhu, "Unleashing the Potential of Prompt Engineering for Large Language Models," Guangdong Provincial Key Laboratory of Interdisciplinary Research and Application for Data Science, BNU-HKBU United International College, Zhuhai, Guangdong, China, arXiv preprint arXiv:2310.14735v6, May 2025.

[6] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. "Language models are unsupervised multitask learners." OpenAI blog, 1(8):9, 2019.

[7] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. "Language models are few-shot learners", arXiv:2005.14165v4 [cs.CL] 22 Jul 2020.

[8] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. Advances in Neural Information Processing Systems, arXiv:2201.11903v6 [cs.CL] 10 Jan 2023.

[9] Zhuosheng Zhang, Aston Zhang, Mu Li, and Alex Smola. Automatic chain of thought prompting in large language models. arXiv preprint arXiv:2210.03493, 2022.

[10] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha

Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. arXiv preprint arXiv:2203.11171, 2022.

[11] Yew Ken Chia, Guizhen Chen, Luu Anh Tuan, Soujanya Poria, and Lidong Bing. Contrastive chain-of-thought prompting. arXiv preprint arXiv:2311.09277, 2023.

[12] Rajasekhar Reddy Mekala, Yasaman Razeghi, and Sameer Singh. Echoprompt: Instructing the model to rephrase queries for improved in-context learning, 2024.Rajasekhar Reddy Mekala, Yasaman Razeghi, and Sameer Singh. Echoprompt: Instructing the model to rephrase queries for improved in-context learning, 2024.

[13] Zhanke Zhou, Rong Tao, Jianing Zhu, Yiwen Luo, Zengmao Wang, and Bo Han. Can language models perform robust reasoning in chain-of-thought prompting with noisy rationales? In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, Advances in Neural Information Processing Systems, volume 37, pages 123846–123910. Curran Associates, Inc., 2024.

[14] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. arXiv preprint arXiv:2305.10601, 2023.

[15] Yao Yao, Zuchao Li, and Hai Zhao. Beyond chain-of-thought, effective graph-of-thought reasoning in large language models. arXiv preprint arXiv:2305.16582, 2023.

[16] Ling Yang, Zhaochen Yu, Tianjun Zhang, Shiyi Cao, Minkai Xu, Wentao Zhang, Joseph E Gonzalez, and Bin Cui. Buffer of thoughts: Thought-augmented reasoning with large language models. Advances in Neural Information Processing Systems, 2024.

[17] Wachara Fungwacharakorn, Nguyen Ha Thanh, May Myo Zin, and Ken Satoh. Layer-of-thoughts prompting (lot): Leveraging llm-based retrieval with constraint hierarchies, 2024.

[18] Xinliang Frederick Zhang, Nick Beauchamp, and Lu Wang. Narrative-of-thought: Improving temporal reasoning of large language models via recounted narratives, 2024.

[19] Tongxuan Liu, Wenjiang Xu, Weizhe Huang, Xingyu Wang, Jiaxing Wang, Hailong Yang, and Jing Li. Logic-of-thought: Injecting logic into contexts for full reasoning in large language models, 2024.

[20] Yucheng Zhou, Xiubo Geng, Tao Shen, Chongyang Tao, Guodong Long, Jian-Guang Lou, and Jianbing Shen. Thread of thought unraveling chaotic contexts. arXiv preprint arXiv:2311.08734, 2023.

[21] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Prasad Majumder, Shashank Gupta, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023.

[22] Jason Weston and Sainbayar Sukhbaatar. System 2 attention (is something you might need too). arXiv preprint arXiv:2311.11829, 2023.

[23] Zilong Wang, Hao Zhang, Chun-Liang Li, Julian Martin Eisenschlos, Vincent Perot, Zifeng Wang, Lesly Miculicich, Yasuhisa Fujii, Jingbo Shang, Chen-Yu Lee, and Tomas Pfister. Chain-of-table: Evolving tables in the reasoning chain for table understanding, 2024.

[24] Xufeng Zhao, Mengdi Li, Wenhao Lu, Cornelius Weber, Jae Hee Lee, Kun Chu, and Stefan Wermter. Enhancing zero-shot chain-of-thought reasoning in large language models through logic, 2023.

[25] Hanxu Hu, Hongyuan Lu, Huajian Zhang, Yun-Ze Song, Wai Lam, and Yue Zhang. Chain-of-symbol prompting elicits planning in large langauge models, 2023.

[26] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. Advances in Neural Information Processing Systems, 33:9459–9474, 2020.

[27] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. arXiv preprint arXiv:2210.03629, 2022.

[28] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. Chain-of-verification reduces hallucination in large language models. arXiv preprint arXiv:2309.11495, 2023.

[29] Wenhao Yu, Hongming Zhang, Xiaoman Pan, Kaixin Ma, Hongwei Wang, and Dong Yu. Chain-of-note: Enhancing robustness in retrieval-augmented language models, 2023.

[30] Xingxuan Li, Ruochen Zhao, Yew Ken Chia, Bosheng Ding, Shafiq Joty, Soujanya Poria, and Lidong Bing. Chain-of-knowledge: Grounding large language models via dynamic knowledge adapting over heterogeneous sources, 2023.

[31] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. arXiv preprint arXiv:2211.12588, 2022.

[32] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. Structured chain-of-thought prompting for code generation. arXiv preprint arXiv:2305.06599, 2023.

[33] Chengshu Li, Jacky Liang, Andy Zeng, Xinyun Chen, Karol Hausman, Dorsa Sadigh, Sergey Levine, Li Fei-Fei, Fei Xia, and Brian Ichter. Chain of code: Reasoning with a language model-augmented code emulator. arXiv preprint arXiv:2312.04474, 2023.

[34] Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, et al. Show your work: Scratchpads for intermediate computation with language models. arXiv preprint arXiv:2112.00114, 2021.

[35] Haritz Puerto, Martin Tutek, Somak Aditya, Xiaodan Zhu, and Iryna Gurevych. Code prompting elicits conditional reasoning abilities in text+code llms, 2024.

[36] Shizhe Diao, Pengcheng Wang, Yong Lin, and Tong Zhang. Active prompting with chain-of-thought for large language models. arXiv preprint arXiv:2302.12246, 2023.

[37] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. arXiv preprint arXiv:2211.01910, 2022.

[38] Xiaosong Yuan, Chen Shen, Shaotian Yan, Xiaofeng Zhang, Liang Xie, Wenxiao Wang, Renchu

Guan, Ying Wang, and Jieping Ye. Instance-adaptive zero-shot chain-of-thought prompting, 2024.

[39] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. arXiv preprint arXiv:2309.03409, 2023.

[40] Yihe Deng, Weitong Zhang, Zixiang Chen, and Quanquan Gu. Rephrase and respond: Let large language models ask better questions for themselves. arXiv preprint arXiv:2311.04205, 2023.

[41] Huaixiu Steven Zheng, Swaroop Mishra, Xinyun Chen, Heng-Tze Cheng, Ed H Chi, Quoc V Le, and Denny Zhou. Take a step back: evoking reasoning via abstraction in large language models. arXiv preprint arXiv:2310.06117, 2023.

[42] Cheng Li, Jindong Wang, Yixuan Zhang, Kaijie Zhu, Wenxin Hou, Jianxun Lian, Fang Luo, Qiang Yang, and Xing Xie. Large language models understand and can be enhanced by emotional stimuli. arXiv preprint arXiv:2307.11760, 2023.

[43] Linger Deng, Yuliang Liu, Bohan Li, Dongliang Luo, Liang Wu, Chengquan Zhang, Pengyuan Lyu, Ziyang Zhang, Gang Zhang, Errui Ding, Yingying Zhu, and Xiang Bai. R-cot: Reverse chain-of-thought problem generation for geometric reasoning in large multimodal models, 2024.

[44] Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. Chain of draft: Thinking faster by writing less, 2025.

[45] Bin Hong, Jinze Wu, Jiayu Liu, Liang Ding, Jing Sha, Kai Zhang, Shijin Wang, and Zhenya Huang. End-to-end graph flattening method for large language models, 2024.

[46] Bhargavi Paranjape, Scott Lundberg, Sameer Singh, Hannaneh Hajishirzi, Luke Zettlemoyer, and Marco Tulio Ribeiro. Art: Automatic multi-step reasoning and tool-use for large language models. arXiv preprint arXiv:2303.09014, 2023.

[47] Zhengyan Shi, Giuseppe Castellucci, Simone Filice, Saar Kuzi, Elad Kravi, Eugene Agichtein, Oleg Rokhlenko, and Shervin Malmasi. 2025. Ambiguity detection and uncertainty calibration for question answering with large language models. In Proceedings of the 5th Workshop on Trustworthy NLP (TrustNLP 2025).

[48] Chenyang Yang, Yike Shi, Qianou Ma, Michael Xieyang Liu, Christian Kästner, and Tongshuang Wu. 2025. What Prompts Don't Say: Understanding and Managing Underspecification in LLM Prompts. arXiv preprint arXiv:2505.13360 (2025).

[49] Maria Teresa Colangelo, Stefano Guizzardi, Marco Meleti, Elena Calciolari, and Carlo Galli. 2025. How to Write Effective Prompts for Screening Biomedical Literature Using Large Language Models. BioMedInformatics 5, 1 (2025).

[50] Shirley Wu, Michel Galley, Baolin Peng, Hao Cheng, Gavin Li, Yao Dou, Weixin Cai, James Zou, Jure Leskovec, and Jianfeng Gao. 2025. CollabLLM: From Passive Responders to Active Collaborators. arXiv preprint arXiv:2502.00640 (2025).

[51] Rongwu Xu, Zehan Qi, Zhijiang Guo, Cunxiang Wang, Hongru Wang, Yue Zhang, and Wei Xu. 2024. Knowledge conflicts for llms: A survey. arXiv preprint arXiv:2403.08319 (2024).

[52] Juhee Kim, Woohyuk Choi, and Byoungyoung Lee. 2025. Prompt flow integrity to prevent privilege escalation in llm agents. arXiv preprint arXiv:2503.15547 (2025).

[53] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, et al. 2023. Prompt injection attack against llm-integrated

applications. arXiv preprint arXiv:2306.05499 (2023).

[54] Samuel Gehman, Suchin Gururangan, Maarten Sap, Yejin Choi, and Noah A Smith. 2020. Realtoxicityprompts: Evaluating neural toxic degeneration in language models. arXiv preprint arXiv:2009.11462 (2020).

[55] Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. 431–445.

[56] Yusu Qian, Haotian Zhang, Yinfei Yang, and Zhe Gan. 2024. How easy is it to fool your multimodal llms? an empirical analysis on deceptive prompts. arXiv preprint arXiv:2402.13220 (2024).

[57] Anna Neumann, Elisabeth Kirsten, Muhammad Bilal Zafar, and Jatinder Singh. 2025. Position is Power: System Prompts as a Mechanism of Bias in Large Language Models (LLMs). In Proceedings of the 2025 ACM Conference on Fairness, Accountability, and Transparency.

[58] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the middle: How language models use long contexts. arXiv preprint arXiv:2307.03172 (2023).

[59] Bryan Guan, Tanya Roosta, Peyman Passban, and Mehdi Rezagholizadeh. 2025. The Order Effect: Investigating Prompt Sensitivity to Input Order in LLMs. arXiv preprint arXiv:2502.04134 (2025).

[60] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. 2023. Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I learned to start worrying about prompt formatting. arXiv preprint arXiv:2310.11324 (2023).

[61] Jia He, Mukund Rungta, David Koleczek, Arshdeep Sekhon, Franklin X Wang, and Sadid Hasan. 2024. Does prompt formatting have any impact on llm performance? arXiv preprint arXiv:2411.10541 (2024).

[62] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. Advances in neural information processing systems 35 (2022), 24824–24837.

[63] Xindi Wang, Mahsa Salmani, Parsa Omidi, Xiangyu Ren, Mehdi Rezagholizadeh, and Armaghan Eshaghi. 2024. Beyond the limits: A survey of techniques to extend the context length in large language models. arXiv preprint arXiv:2402.02244 (2024).

[64] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. Advances in neural information processing systems 33 (2020).

[65] Keheng Wang, Feiyu Duan, Peiguang Li, Sirui Wang, and Xunliang Cai. 2025. LLMs Know What They Need: Leveraging a Missing Information Guided Framework to Empower Retrieval-Augmented Generation. In Proceedings of the 31st International Conference on Computational Linguistics. 2379–2400.

[66] Genki Kusano, Kosuke Akimoto, and Kunihiro Takeoka. 2024. Are longer prompts always better?

prompt selection in large language models for recommendation systems. arXiv preprint arXiv:2412.14454 (2024).

[67] Priyan Vaithilingam, Tianyi Zhang, and Elena L Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In Chi conference on human factors in computing systems extended abstracts. 1–7.

[68] Shraddha Barke, Michael B James, and Nadia Polikarpova. 2023. Grounded copilot: How programmers interact with code-generating models. Proceedings of the ACM on Programming Languages 7, OOPSLA1 (2023), 85–111.

[69] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. arXiv preprint arXiv:2310.06839 (2023).

[70] Robert L Logan IV, Ivana Balažević, Eric Wallace, Fabio Petroni, Sameer Singh, and Sebastian Riedel. 2021. Cutting down on prompts and parameters: Simple few-shot learning with language models. arXiv preprint arXiv:2106.13353 (2021).

[71] Hanlin Zhu, Banghua Zhu, and Jiantao Jiao. 2024. Efficient prompt caching via embedding similarity. arXiv preprint arXiv:2402.01173 (2024).

[72] Sania Nayab, Giulio Rossolini, Marco Simoni, Andrea Saracino, Giorgio Buttazzo, Nicolamaria Manes, and Fabrizio Giacomelli. 2024. Concise thoughts: Impact of output length on llm reasoning and cost. arXiv preprint arXiv:2407.19825 (2024).

[73] Zhenpeng Chen, Chong Wang, Weisong Sun, Guang Yang, Xuanzhe Liu, Jie M Zhang, and Yang Liu. 2025. Promptware Engineering: Software Engineering for LLM Prompt Development. arXiv preprint arXiv:2503.02400 (2025).

[74] Zhenchang Xing, Qing Huang, Yu Cheng, Liming Zhu, Qinghua Lu, and Xiwei Xu. 2023. Prompt sapper: Llm-empowered software engineering infrastructure for ai-native services. arXiv preprint arXiv:2306.02230 (2023).

[75] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C Schmidt. 2023. A prompt pattern catalog to enhance prompt engineering with chatgpt. arXiv preprint arXiv:2302.11382 (2023).

[76] Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. 2025. Benchmarking and defending against indirect prompt injection attacks on large language models. In Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1. 1809–1820.

[77] Leon Derczynski, Erick Galinkin, Jeffrey Martin, Subho Majumdar, and Nanna Inie. 2024. garak: A framework for security probing large language models. arXiv preprint arXiv:2406.11036 (2024).

[78] Chetan Pathade. 2025. Red teaming the mind of the machine: A systematic evaluation of prompt injection and jailbreak vulnerabilities in llms. arXiv preprint arXiv:2505.04806 (2025).

[79] Yuchen Shao, Yuheng Huang, Jiawei Shen, Lei Ma, Ting Su, and Chengcheng Wan. 2024. Are LLMs Correctly Integrated into Software Systems? arXiv preprint arXiv:2407.05138 (2024).

[80] Schulhoff, S., Ilie, M., Balepur, N., Kahadze, K., Liu, A., Si, C., ... & Resnik, P. (2025). The Prompt Report: A Systematic Survey of Prompt Engineering Techniques. University of Maryland. arXiv

preprint arXiv:2406.06608v6.

[81] Chen, B., Zhang, Z., Langrené, N., & Zhu, S. (2025). Unleashing the Potential of Prompt Engineering for Large Language Models. Guangdong Provincial Key Laboratory of Interdisciplinary Research and Application for Data Science, BNU-HKBU United International College. arXiv preprint arXiv:2310.14735v6.

[82] Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., & Neubig, G. (2023). Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. ACM Computing Surveys, 55(9), 1-35.

[83] Papineni, K., Roukos, S., Ward, W. J., & Zhu, W. J. (2002). BLEU: A Method for Automatic Evaluation of Machine Translation. In Proceedings of the 40th Annual Meeting on Association for Computational Linguistics (pp. 311-318).

[84] Lin, C. Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. In Text Summarization Branches Out (pp. 74-81).

[85] Banerjee, S., & Lavie, A. (2005). METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments. In Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization (pp. 65-72).

[86] Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2020). BERTScore: Evaluating Text Generation with BERT. In Ninth International Conference on Learning Representations.

[87] Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., & Steinhardt, J. (2021). Measuring Massive Multitask Language Understanding. In International Conference on Learning Representations (ICLR).

[88] Talmor, A., Herzig, J., Lourie, N., & Berant, J. (2019). CommonsenseQA: A Question Answering Challenge Targeting Commonsense Knowledge. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (pp. 4149-4158).

[89] Lin, Y. T., & Chen, Y. N. (2023). LLM-eval: Unified Multi-dimensional Automatic Evaluation for Open-domain Conversations with Large Language Models. arXiv preprint arXiv:2305.13711.

[90] Liu, Y., Zhang, T., & Liang, P. (2023). GPTEval: NLG Evaluation using GPT-4 with Better Human Alignment. arXiv preprint arXiv:2303.16634.

[91] ai, A. B., Mohankumar, A. K., & Khapra, M. M. (2022). A survey of evaluation metrics used for NLG systems. ACM Computing Surveys (CSUR), 55(2), 1-39.

[92] Chen, Y., Gao, S., Pan, Y., Tao, Y., & Mei, T. (2023). Exploring the use of large language models for reference-free text quality evaluation: An empirical study. In Findings of the Association for Computational Linguistics: IJCNLP-AACL 2023 (Findings) (pp. 361-374).

[93] Chan, C. M., Chen, W., Su, Y., Yu, J., Xue, W., Zhang, S., & Liu, Z. (2024). ChatEval: Towards Better LLM-based Evaluators through Multi-agent Debate. In The Twelfth International Conference on Learning Representations.